

SOLIDWORKS API (.NET VBA C#) — Examples (35/50):

Get_Visible_Drawing_Components_Example_V-

Ignore_Feature_Colors_Example_VB

SOLIDWORKS API Documentation (converted from CHM)

Get Visible Drawing Components Example (VB.NET)

This example shows how to get the visible drawing components in a drawing of an assembly.

```
'-----
' Preconditions:
' 1. Open a drawing document of an assembly that contains Drawing View1
'    with one or more visible components.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets the visible components and their names.
' 2. Examine the Immediate window.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swView As View
        Dim swDrComp As DrawingComponent
        Dim swComp As Component2
        Dim vComps() As Object
        Dim vBodies As Object = Nothing
        Dim boolstatus As Boolean
        Dim itr As Integer

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager
        boolstatus = swModel.ActivateView("Drawing View1")
        boolstatus = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
        swView = swSelMgr.GetSelectedObject6(1, -1)
        swModel.ClearSelection2(True)

        'Get all visible components in Drawing View1
        vComps = swView.GetVisibleDrawingComponents

        'Print the number of visible components
        Debug.Print("Number of visible components:")
        Debug.Print("  " & vComps.Length)

        'Iterate through the visible components and print each one's name
        Debug.Print("Names of visible components:")
        For itr = 0 To UBound(vComps)
            swDrComp = vComps(itr)
            Debug.Print("  " & swDrComp.Name)
            ' Print the referenced assembly component name
            swComp = swDrComp.Component
            If Not IsNothing(swComp) Then
                Debug.Print("    " & swComp.Name2)
                Call swComp.GetBodies3(swBodyType_e.swAllBodies, vBodies)
                If Not IsNothing(vBodies) Then
                    Debug.Print("      Component bodies exist.")
                End If
            End If
        Next
    End Sub
End Class
```

```
        End If
    End If
Next itr

End Sub
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class
```

Get Weld Bead Symbol Data Example (C#)

This example shows how to get weld bead symbol data in a drawing.

```
//-----
// Preconditions:
// 1. Open a drawing that contains at least one view with a weld bead
//    annotation.
// 2. Open an Immediate window.
//
// Postconditions: Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace WeldBead_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DrawingDoc swDraw = default(DrawingDoc);
            Sheet swSheet = default(Sheet);
            View swView = default(View);
            WeldBead swWBead = default(WeldBead);
            DisplayData swDispData = default(DisplayData);
            string strFilled = null;
            int lineCount = 0;
            double[] vLine = null;
            int lineColor = 0;
            int lineLineType = 0;
            int lineStyle = 0;
            int lineWeight = 0;
            double lineX = 0;
            double lineY = 0;
            double lineZ = 0;
            int polygonCount = 0;
            double[] vPolygon = null;
            int polyColor = 0;
            int polyLineType = 0;
            int polyNumPts = 0;
            double polyX = 0;
            double polyY = 0;
            double polyZ = 0;
            int index = 0;
            int index2 = 0;
            int arrayIndex = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swDraw = (DrawingDoc)swModel;
            swSheet = (Sheet)swDraw.GetCurrentSheet();
            swView = (View)swDraw.GetFirstView();

            while ((swView != null))
            {
                Debug.Print(swView.Name);
                swWBead = swView.GetFirstWeldBead();

                while ((swWBead != null))
                {
                    Debug.Print(" Weld Bead...");
                    if ((swWBead.SolidFill == false))
                    {
                        strFilled = "open";
                    }
                    else
                    {
                        strFilled = "solid filled";
                    }

                    swDispData = (DisplayData)swWBead.GetAnnotation().GetDisplayData();
                    lineCount = swDispData.GetLineCount();
                    Debug.Print(" Line count = " + lineCount);

                    for (index = 0; index <= lineCount - 1; index++)
                    {
                        vLine = (double[])swDispData.GetLineAtIndex3(index);
                        if ((vLine != null))
                        {

```

```

        lineColor = (int)vLine[0];
        lineLineType = (int)vLine[1];
        lineStyle = (int)vLine[2];
        lineWeight = (int)vLine[3];
        Debug.Print("        Color = " + lineColor + ", line type = " + lineLineType + ", line style = " + lineSty

        arrayIndex = 4;
        for (index2 = 0; index2 <= 1; index2++)
        {
            lineX = (double)vLine[arrayIndex];
            lineY = (double)vLine[arrayIndex + 1];
            lineZ = (double)vLine[arrayIndex + 2];
            Debug.Print("        " + index2 + " (" + lineX + ", " + lineY + ", " + lineZ + ")");
            arrayIndex = arrayIndex + 3;
        }
    }

    Debug.Print("        Polyline count = " + swDispData.GetPolyLineCount());
    Debug.Print("        Arc count = " + swDispData.GetArcCount());
    polygonCount = swDispData.GetPolygonCount();
    Debug.Print("        Polygon count = " + polygonCount);

    for (index = 0; index <= polygonCount - 1; index++)
    {
        vPolygon = (double[])swDispData.GetPolygonAtIndex(index);
        if (vPolygon != null)
        {
            polyColor = (int)vPolygon[0];
            polyLineType = (int)vPolygon[1];
            polyNumPts = (int)vPolygon[4];
            Debug.Print("        Color = " + polyColor + ", line type = " + polyLineType + ", point count = " + polyNu
            arrayIndex = 5;

            for (index2 = 0; index2 <= polyNumPts - 1; index2++)
            {
                polyX = (double)vPolygon[arrayIndex];
                polyY = (double)vPolygon[arrayIndex + 1];
                polyZ = (double)vPolygon[arrayIndex + 2];
                Debug.Print("        " + index2 + " (" + polyX + ", " + polyY + ", " + polyZ + ")");
                arrayIndex = arrayIndex + 3;
            }
        }
    }
    swWBead = swWBead.GetNext();
}
swView = (View)swView.GetNextView();
}

}

public SldWorks swApp;

}

}

```

Get Weld Bead Symbol Data Example (VBA)

This example shows how to get weld bead symbol data in a drawing.

```

'-----
' Preconditions:
' 1. Open a drawing that contains at least one view with a weld bead
'   annotation.
' 2. Open an Immediate window.
'
' Postconditions: Inspect the Immediate window.
'-----

```

Option Explicit

```
Sub main()
```

```

    Dim swApp                As SldWorks.SldWorks
    Dim swModel              As SldWorks.ModelDoc2
    Dim swDraw               As SldWorks.DrawingDoc
    Dim swSheet              As SldWorks.Sheet
    Dim swView               As SldWorks.View
    Dim swWBead              As SldWorks.WeldBead
    Dim swDispData           As SldWorks.DisplayData
    Dim strFilled As String
    Dim lineCount As Long
    Dim vLine As Variant
    Dim lineColor As Long, lineLineType As Long, lineStyle As Long, lineWeight As Long
    Dim lineX As Double, lineY As Double, lineZ As Double
    Dim polygonCount As Long
    Dim vPolygon As Variant
    Dim polyColor As Long, polyLineType As Long, polyNumPts As Long
    Dim polyX As Double, polyY As Double, polyZ As Double
    Dim index As Long, index2 As Long, arrayIndex As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel
    Set swSheet = swDraw.GetCurrentSheet
    Set swView = swDraw.GetFirstView

    While Not swView Is Nothing
        Debug.Print swView.Name
        Set swWBead = swView.GetFirstWeldBead()

        While Not swWBead Is Nothing
            Debug.Print " Weld Bead..."
            If (swWBead.SolidFill = False) Then
                strFilled = "open"
            Else
                strFilled = "solid filled"
            End If

            Set swDispData = swWBead.GetAnnotation().GetDisplayData()
            lineCount = swDispData.GetLineCount
            Debug.Print " Line count = " & lineCount

            For index = 0 To lineCount - 1
                vLine = swDispData.GetLineAtIndex3(index)
                If Not IsEmpty(vLine) Then
                    lineColor = vLine(0)
                    lineLineType = vLine(1)
                    lineStyle = vLine(2)
                    lineWeight = vLine(3)
                    Debug.Print " Color = " & lineColor & ", line type = " & lineLineType & ", line style = " & lineStyle & ", " & lineWeight
                    arrayIndex = 4
                    For index2 = 0 To 1
                        lineX = vLine(arrayIndex)
                        lineY = vLine(arrayIndex + 1)
                        lineZ = vLine(arrayIndex + 2)
                        Debug.Print " " & index2 & " (" & lineX & ", " & lineY & ", " & lineZ & ")"
                        arrayIndex = arrayIndex + 3
                    Next index2
                End If
            Next index

            Debug.Print " Polyline count = " & swDispData.GetPolyLineCount
            Debug.Print " Arc count = " & swDispData.GetArcCount
            polygonCount = swDispData.GetPolygonCount
            Debug.Print " Polygon count = " & polygonCount

            For index = 0 To polygonCount - 1
                vPolygon = swDispData.GetPolygonAtIndex(index)
                If Not IsEmpty(vPolygon) Then
                    polyColor = vPolygon(0)
                    polyLineType = vPolygon(1)
                    polyNumPts = vPolygon(4)
                    Debug.Print " Color = " & polyColor & ", line type = " & polyLineType & ", point count = " & polyNumPts & ", " & strFilled
                    arrayIndex = 5

```

```

        For index2 = 0 To polyNumPts - 1
            polyX = vPolygon(arrayIndex)
            polyY = vPolygon(arrayIndex + 1)
            polyZ = vPolygon(arrayIndex + 2)
            Debug.Print "          " & index2 & " (" & polyX & ", " & polyY & ", " & polyZ & ")"
            arrayIndex = arrayIndex + 3
        Next index2
    End If
Next index
Set swWBead = swWBead.GetNext()
Wend
Set swView = swView.GetNextView
Wend
End Sub

```

Get Weld Bead Symbol Data Example (VB.NET)

This example shows how to get weld bead symbol data in a drawing.

```

'-----
' Preconditions:
' 1. Open a drawing that contains at least one view with a weld bead
'    annotation.
' 2. Open an Immediate window.
' Postconditions: Inspect the Immediate window.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub main()

        Dim swModel As ModelDoc2
        Dim swDraw As DrawingDoc
        Dim swSheet As Sheet
        Dim swView As View
        Dim swWBead As WeldBead
        Dim swDispData As DisplayData
        Dim strFilled As String
        Dim lineCount As Integer
        Dim vLine As Object
        Dim lineColor As Integer, lineLineType As Integer, lineStyle As Integer, lineWeight As Integer
        Dim lineX As Double, lineY As Double, lineZ As Double
        Dim polygonCount As Integer
        Dim vPolygon As Object
        Dim polyColor As Integer, polyLineType As Integer, polyNumPts As Integer
        Dim polyX As Double, polyY As Double, polyZ As Double
        Dim index As Integer, index2 As Integer, arrayIndex As Integer

        swModel = swApp.ActiveDoc
        swDraw = swModel
        swSheet = swDraw.GetCurrentSheet
        swView = swDraw.GetFirstView

        While Not swView Is Nothing

            Debug.Print(swView.Name)
            swWBead = swView.GetFirstWeldBead()

            While Not swWBead Is Nothing

                Debug.Print(" Weld Bead...")
                If (swWBead.SolidFill = False) Then
                    strFilled = "open"
                Else
                    strFilled = "solid filled"
                End If

                swDispData = swWBead.GetAnnotation().GetDisplayData()
                lineCount = swDispData.GetLineCount
                Debug.Print(" Line count = " & lineCount)

                For index = 0 To lineCount - 1
                    vLine = swDispData.GetLineAtIndex3(index)
                    If Not IsNothing(vLine) Then
                        lineColor = vLine(0)
                        lineLineType = vLine(1)
                        lineStyle = vLine(2)
                        lineWeight = vLine(3)
                        Debug.Print(" Color = " & lineColor & ", line type = " & lineLineType & ", line style = " & lineStyle & ", line weight = " & lineWeight)
                        arrayIndex = 4
                        For index2 = 0 To 1
                            lineX = vLine(arrayIndex)
                            lineY = vLine(arrayIndex + 1)
                            lineZ = vLine(arrayIndex + 2)
                            Debug.Print(" " & index2 & " (" & lineX & ", " & lineY & ", " & lineZ & ")")
                            arrayIndex = arrayIndex + 3
                        Next index2
                    End If
                Next index

                Debug.Print(" Polyline count = " & swDispData.GetPolyLineCount)
                Debug.Print(" Arc count = " & swDispData.GetArcCount)
                polygonCount = swDispData.GetPolygonCount
                Debug.Print(" Polygon count = " & polygonCount)

                For index = 0 To polygonCount - 1
                    vPolygon = swDispData.GetPolygonAtIndex(index)
                    If Not IsNothing(vPolygon) Then

```

```

        polyColor = vPolygon(0)
        polyLineType = vPolygon(1)
        polyNumPts = vPolygon(4)
        Debug.Print("        Color = " & polyColor & ", line type = " & polyLineType & ", point count = " & polyNumPts)
        arrayIndex = 5

        For index2 = 0 To polyNumPts - 1
            polyX = vPolygon(arrayIndex)
            polyY = vPolygon(arrayIndex + 1)
            polyZ = vPolygon(arrayIndex + 2)
            Debug.Print("        " & index2 & " (" & polyX & ", " & polyY & ", " & polyZ & ")")
            arrayIndex = arrayIndex + 3
        Next index2
    End If
Next index
swWBead = swWBead.GetNext()
End While
swView = swView.GetNextView
End While

End Sub

Public swApp As SldWorks

End Class

```

Get Weldment Cut List Feature and Annotations Example (C#)

This example shows how to get a weldment cut list feature and weldment cut list annotations.

```
//-----
// Preconditions:
// 1. Open a Drawing document that has a weldment cut list table.
// 2. Open an Immediate window.
//
// Postconditions: Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace GetWeldmentCutListFeature_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        Feature swFeature;
        WeldmentCutListFeature swWeldmentCutListFeat;
        object[] vWeldCutListAnnotations;
        object[] vCustProperties;
        int i;
        int j;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Traverse FeatureManager design tree
            // Get first feature in FeatureManager design tree
            swFeature = (Feature)swModel.FirstFeature();

            // If the type of feature is "WeldmentTableFeat"
            // then get the WeldmentCutListFeature object
            while ((swFeature != null))
            {
                if (swFeature.GetTypeName() == "WeldmentTableFeat")
                {
                    Debug.Print(swFeature.Name);
                    swWeldmentCutListFeat = (WeldmentCutListFeature)swFeature.GetSpecificFeature2();
                    Debug.Print("   Name of configuration: " + swWeldmentCutListFeat.Configuration);
                    Debug.Print("   Keep missing items: " + swWeldmentCutListFeat.KeepCurrentItemNumbers);
                    Debug.Print("   Strikeout missing items: " + swWeldmentCutListFeat.StrikeoutMissingItems);
                    Debug.Print("   Starting sequence number: " + swWeldmentCutListFeat.SequenceStartNumber);

                    // Get the weldment cut list annotations
                    vWeldCutListAnnotations = (object[])swWeldmentCutListFeat.GetTableAnnotations();
                    for (i = vWeldCutListAnnotations.GetLowerBound(0); i <= vWeldCutListAnnotations.GetUpperBound(0); i++)
                    {
                        // Print the number custom properties and then the custom properties
                        Debug.Print("   Number of custom properties: " + ((WeldmentCutListAnnotation)vWeldCutListAnnotations[i]).GetCustPropertiesCount());
                        vCustProperties = (object[])
                            ((WeldmentCutListAnnotation)vWeldCutListAnnotations[i]).GetAllCustomProperties();
                        for (j = vCustProperties.GetLowerBound(0); j <= vCustProperties.GetUpperBound(0); j++)
                        {
                            Debug.Print("       Custom property: " + vCustProperties[j]);
                        }
                    }

                    // Get the next feature in the FeatureManager design tree
                    swFeature = (Feature)swFeature.GetNextFeature();
                }
            }
        }

        public SldWorks swApp;
    }
}
```

Get Weldment Cut List Feature and Annotations Example (VBA)

This example shows how to get a weldment cut list feature and weldment cut list annotations.

```
'-----
' Preconditions:
' 1. Open a drawing document that has a weldment cut list table.
' 2. Open an Immediate window.
'
' Postconditions: Inspect the Immediate window.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swFeature As SldWorks.Feature
Dim swWeldmentCutListFeat As SldWorks.WeldmentCutListFeature
Dim vWeldCutListAnnotations As Variant
Dim vCustProperties As Variant
Dim i As Long
Dim j As Long

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    ' Traverse FeatureManager design tree
    ' Get first feature in FeatureManager design tree
    Set swFeature = swModel.FirstFeature

    ' If the type of feature is "WeldmentTableFeat"
    ' then get the WeldmentCutListFeature object
    Do While Not swFeature Is Nothing

        If swFeature.GetTypeName = "WeldmentTableFeat" Then
            Debug.Print swFeature.Name
            Set swWeldmentCutListFeat = swFeature.GetSpecificFeature2
            Debug.Print " Name of configuration: " & swWeldmentCutListFeat.Configuration
            Debug.Print " Keep missing items: " & swWeldmentCutListFeat.KeepCurrentItemNumbers
            Debug.Print " Strike out missing items: " & swWeldmentCutListFeat.StrikeoutMissingItems
            Debug.Print " Starting sequence number: " & swWeldmentCutListFeat.SequenceStartNumber

            ' Get the weldment cut list annotations
            vWeldCutListAnnotations = swWeldmentCutListFeat.GetTableAnnotations
            For i = LBound(vWeldCutListAnnotations) To UBound(vWeldCutListAnnotations)
                ' Print the number custom properties and then the custom properties
                Debug.Print " Number of custom properties: " & vWeldCutListAnnotations(i).GetAllCustomPropertiesCount
                vCustProperties = vWeldCutListAnnotations(i).GetAllCustomProperties
                For j = LBound(vCustProperties) To UBound(vCustProperties)
                    Debug.Print " Custom property: " & vCustProperties(j)
                Next j
            Next i
        End If

        ' Get the next feature in the FeatureManager design tree
        Set swFeature = swFeature.GetNextFeature
    Loop
End Sub
```

Get Weldment Cut List Feature and Annotations Example (VB.NET)

This example shows how to get a weldment cut list feature and weldment cut list annotations.

```

'-----
' Preconditions:
' 1. Open a Drawing document that has a weldment cut list table.
' 2. Open an Immediate window.
'-----
' Postconditions: Inspect the Immediate window.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swFeature As Feature
    Dim swWeldmentCutListFeat As WeldmentCutListFeature
    Dim vWeldCutListAnnotations As Object
    Dim vCustProperties As Object
    Dim i As Integer
    Dim j As Integer

    Sub main()

        swModel = swApp.ActiveDoc

        ' Traverse FeatureManager design tree
        ' Get first feature in FeatureManager design tree
        swFeature = swModel.FirstFeature

        ' If the type of feature is "WeldmentTableFeat"
        ' then get the WeldmentCutListFeature object
        Do While Not swFeature Is Nothing

            If swFeature.GetTypeName = "WeldmentTableFeat" Then
                Debug.Print(swFeature.Name)
                swWeldmentCutListFeat = swFeature.GetSpecificFeature2
                Debug.Print("    Name of configuration: " & swWeldmentCutListFeat.Configuration)
                Debug.Print("    Keep missing items: " & swWeldmentCutListFeat.KeepCurrentItemNumbers)
                Debug.Print("    Strikeout missing items: " & swWeldmentCutListFeat.StrikeoutMissingItems)
                Debug.Print("    Starting sequence number: " & swWeldmentCutListFeat.SequenceStartNumber)

                ' Get the weldment cut list annotations
                vWeldCutListAnnotations = swWeldmentCutListFeat.GetTableAnnotations
                For i = LBound(vWeldCutListAnnotations) To UBound(vWeldCutListAnnotations)
                    ' Print the number custom properties and then the custom properties
                    Debug.Print("    Number of custom properties: " & vWeldCutListAnnotations(i).GetAllCustomPropertiesCount)
                    vCustProperties = vWeldCutListAnnotations(i).GetAllCustomProperties
                    For j = LBound(vCustProperties) To UBound(vCustProperties)
                        Debug.Print("        Custom property: " & vCustProperties(j))
                    Next j
                Next i
            End If

            ' Get the next feature in the FeatureManager design tree
            swFeature = swFeature.GetNextFeature

        Loop

    End Sub

    Public swApp As SldWorks

End Class

```

Get Weldment Trim Extend Corner Type Example (C#)

This example shows how to get the type of corner used for a weldment trim extend feature.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\weldment_box3.sldprt.
// 2. Open an Immediate window.
//
// Postconditions: Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace GetWeldmentTrimExtendCornerType_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        Feature swFeature;
        WeldmentTrimExtendFeatureData swWeldmentTrimExtend;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;

            //Traverse FeatureManager design tree
            //Get first feature in FeatureManager design tree
            swFeature = (Feature)swModel.FirstFeature();

            //If the type of feature is "WeldCornerFeat" then get the
            //WeldmentTrimExtendFeatureData object and then get the type of corner
            while ((swFeature != null))
            {
                if (swFeature.GetTypeName() == "WeldCornerFeat")
                {
                    Debug.Print(swFeature.Name);
                    swWeldmentTrimExtend = (WeldmentTrimExtendFeatureData)swFeature.GetDefinition();
                    Debug.Print(swWeldmentTrimExtend.CornerType.ToString());
                }

                //Get the next feature in the FeatureManager design tree
                swFeature = (Feature)swFeature.GetNextFeature();
            }
        }

        public SldWorks swApp;
    }
}
```

Get Weldment Trim Extend Corner Type Example (VBA)

This example shows how to get the type of corner used for a weldment trim extend feature.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\weldment_box3.sldprt.  
' 2. Open an Immediate window.  
'-----  
' Postconditions: Inspect the Immediate window.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swFeature As SldWorks.Feature  
Dim swWeldmentTrimExtend As SldWorks.WeldmentTrimExtendFeatureData  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    'Traverse FeatureManager design tree  
    'Get first feature in FeatureManager design tree  
    Set swFeature = swModel.FirstFeature  
  
    'If the type of feature is "WeldCornerFeat" then get the  
    'WeldmentTrimExtendFeatureData object and then get the type of corner  
    Do While Not swFeature Is Nothing  
        If swFeature.GetType = "WeldCornerFeat" Then  
            Debug.Print swFeature.Name  
            Set swWeldmentTrimExtend = swFeature.GetDefinition  
            Debug.Print swWeldmentTrimExtend.CornerType  
        End If  
  
        'Get the next feature in the FeatureManager design tree  
        Set swFeature = swFeature.GetNextFeature  
    Loop  
End Sub
```

Get Weldment Trim Extend Corner Type Example (VB.NET)

This example shows how to get the type of corner used for a weldment trim extend feature.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\weldment_box3.sldprt.
' 2. Open an Immediate window.
'-----
' Postconditions: Inspect the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swFeature As Feature
    Dim swWeldmentTrimExtend As WeldmentTrimExtendFeatureData

    Sub main()

        swModel = swApp.ActiveDoc

        'Traverse FeatureManager design tree
        'Get first feature in FeatureManager design tree
        swFeature = swModel.FirstFeature

        'If the type of feature is "WeldCornerFeat" then get the
        'WeldmentTrimExtendFeatureData object and then get the type of corner
        Do While Not swFeature Is Nothing
            If swFeature.GetTypeName = "WeldCornerFeat" Then
                Debug.Print(swFeature.Name)
                swWeldmentTrimExtend = swFeature.GetDefinition
                Debug.Print(swWeldmentTrimExtend.CornerType)
            End If

            'Get the next feature in the FeatureManager design tree
            swFeature = swFeature.GetNextFeature
        Loop

    End Sub

    Public swApp As SldWorks

End Class

```

Get What's Wrong Example (C#)

This example shows how to get the What's Wrong information for a document.

```
//-----
//
// Preconditions: Model document is active. Examine the Immediate
//      window after running this macro to see the What's Wrong
//      items in the model document.
//
// Postconditions: None
//
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace GetWhatsWrongCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            object oFeatures;
            object oErrorCodes;
            object oWarnings;
            object[] Features = null;
            int[] ErrorCodes = null;
            bool[] Warnings = null;
            bool boolstatus = false;
            int i = 0;
            int nbrWhatsWrong = 0;
            Feature swFeature = default(Feature);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            nbrWhatsWrong = swModelDocExt.GetWhatsWrongCount();
            Debug.Print("Number of What's Wrong items: " + nbrWhatsWrong);
            Debug.Print("");
            if (nbrWhatsWrong > 0) {
                boolstatus = swModelDocExt.GetWhatsWrong(out oFeatures, out oErrorCodes, out oWarnings);
                Features = (object[])oFeatures;
                ErrorCodes = (int[])oErrorCodes;
                Warnings = (bool[])oWarnings;
                for (i = 0; i < Features.Length; i++) {
                    swFeature = (Feature)Features[i];
                    Debug.Print(" Name of feature: " + swFeature.GetTypeName2());
                    Debug.Print(" Error: " + ErrorCodes[i]);
                    Debug.Print(" Did SOLIDWORKS Design flag this item as a warning ? " + Warnings[i]);
                    Debug.Print("");
                }
            }
            else {
                Debug.Print("No What's Wrong items.");
            }
        }
    }
}
```

```
    }  
  
}  
    /// <summary>  
    /// The SldWorks swApp variable is pre-assigned for you.  
    /// </summary>  
    public SldWorks swApp;  
}  
}
```

Get What's Wrong Example (VBA)

This example shows how to get the What's Wrong information for a document.

```
'-----  
' Preconditions: Model document is active. Examine the Immediate  
'               window after running this macro to see the What's Wrong  
'               items in the model document.  
'  
' Postconditions: None  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim vFeatures As Variant  
Dim vErrorCodes As Variant  
Dim vWarnings As Variant  
Dim boolstatus As Boolean  
Dim i As Long  
Dim nbrWhatsWrong As Long  
Dim swFeature As SldWorks.Feature  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swModelDocExt = swModel.Extension  
nbrWhatsWrong = swModelDocExt.GetWhatsWrongCount  
Debug.Print "Number of What's Wrong items: " & nbrWhatsWrong  
Debug.Print ""  
If nbrWhatsWrong > 0 Then  
    boolstatus = swModelDocExt.GetWhatsWrong(vFeatures, vErrorCodes, vWarnings)  
    For i = 0 To UBound(vFeatures)  
        Set swFeature = vFeatures(i)  
        Debug.Print "  Name of feature: " & swFeature.GetTypeName2  
        Debug.Print "  Error: " & vErrorCodes(i)  
        Debug.Print "  Did SOLIDWORKS Design flag this item as a warning ? " & vWarnings(i)  
        Debug.Print ""  
    Next i  
Else  
    Debug.Print "No What's Wrong items."  
End If  
  
End Sub
```

Get What's Wrong Example (VB.NET)

This example shows how to get the What's Wrong information for a document.

```
'-----
' Preconditions: Model document is active. Examine the Immediate
'               window after running this macro to see the What's Wrong
'               items in the model document.
'
' Postconditions: None
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim vFeatures As Object = Nothing
        Dim vErrorCodes As Object = Nothing
        Dim vWarnings As Object = Nothing
        Dim boolstatus As Boolean
        Dim i As Integer
        Dim nbrWhatsWrong As Integer
        Dim swFeature As Feature

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        nbrWhatsWrong = swModelDocExt.GetWhatsWrongCount
        Debug.Print("Number of What's Wrong items: " & nbrWhatsWrong)
        Debug.Print("")
        If nbrWhatsWrong > 0 Then
            boolstatus = swModelDocExt.GetWhatsWrong(vFeatures, vErrorCodes, vWarnings)
            For i = 0 To UBound(vFeatures)
                swFeature = vFeatures(i)
                Debug.Print("  Name of feature: " & swFeature.GetTypeName2)
                Debug.Print("  Error: " & vErrorCodes(i))
                Debug.Print("  Did SOLIDWORKS Design flag this item as a warning ? " & vWarnings(i))
                Debug.Print("")
            Next i
        Else
            Debug.Print("No What's Wrong items.")
        End If
    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Get Where Annotations Can Be Shown Example (C#)

This example shows how to get the names of the annotation views where an annotation can be shown.

```
//-----
// Preconditions:
// 1. Open a part document with one or more annotations
//   in one or more annotation views.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Gets the annotation views.
// 2. Iterates through the annotation views and annotations.
// 3. Prints the name of each annotation view and number of
//   annotations in that annotation view.
// 4. Prints whether each annotation in that annotation view
//   can be shown in the other annotation views and in
//   multiple annotation views.
// 5. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace CanShowCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelExt = default(ModelDocExtension);
            object[] swAnnViews = null;
            object[] annotations = null;
            AnnotationView swAnnView = default(AnnotationView);
            Annotation swAnn = default(Annotation);
            Feature swFeat = default(Feature);
            int i = 0;
            int j = 0;
            int k = 0;
            int nbrAnnotations = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelExt = (ModelDocExtension)swModel.Extension;

            //Get the annotation views
            swAnnViews = (object[])swModelExt.AnnotationViews;
            for (i = 0; i <= swModelExt.AnnotationViewCount - 1; i++)
            {
                swAnnView = (AnnotationView)swAnnViews[i];
            }
            Debug.Print("Number of annotation views = " + swModelExt.AnnotationViewCount);
            Debug.Print("");

            //Iterate through the annotation views and annotations
            //Print the name of each annotation view and number of annotations
            //In that annotation view
            //Print whether each annotation in that annotation view
            //can be shown in the other annotation views and in
            //multiple annotation views
            Debug.Print(" Name of annotation view and number of annotations in the annotation view ");
            Debug.Print("");
            for (i = 0; i <= swModelExt.AnnotationViewCount - 1; i++)
            {
                swAnnView = (AnnotationView)swAnnViews[i];
                swAnnView.Activate();
                annotations = (object[])swAnnView.GetAnnotations2(false, false);
                swFeat = (Feature)swAnnView;
                nbrAnnotations = swAnnView.AnnotationCount;
                if (nbrAnnotations > 0)
                {
                    Debug.Print("");
                }
                Debug.Print("      " + swFeat.Name + " = " + nbrAnnotations);
                if ((annotations != null))
                {
                    j = 0;
                    for (j = 0; j <= nbrAnnotations - 1; j++)
                    {
                        Debug.Print("          Can show annotation " + (j + 1) + " in these annotation views: ");
                        swAnn = (Annotation)annotations[j];
                        for (k = 0; k <= swModelExt.AnnotationViewCount - 1; k++)
                        {
                            swAnnView = (AnnotationView)swAnnViews[k];
                            swFeat = (Feature)swAnnView;
                            Debug.Print("              " + swFeat.Name + " = " + swAnn.CanShowInAnnotationView(swFeat.Name));
                        }
                        Debug.Print("          multiple = " + swAnn.CanShowInMultipleAnnotationViews());
                        Debug.Print("");
                    }
                }
            }
        }
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
```

Get Where Annotations Can be Shown Example (VBA)

This example shows how to get the names of the annotation views where an annotation can be shown.

```
'-----
' Preconditions:
' 1. Open a part document with one or more annotations
'    in one or more annotation views.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets the annotation views.
' 2. Iterates through the annotation views and annotations.
' 3. Prints the name of each annotation view and number of
'    annotations in that annotation view.
' 4. Prints whether each annotation in that annotation view
'    can be shown in the other annotation views and in
'    multiple annotation views.
' 5. Examine the Immediate window.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelExt As SldWorks.ModelDocExtension
Dim swAnnViews As Variant
Dim annotations As Variant
Dim swAnnView As SldWorks.AnnotationView
Dim swAnn As SldWorks.Annotation
Dim swFeat As SldWorks.Feature
Dim i As Long
Dim j As Long
Dim k As Long
Dim nbrAnnotations As Long

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelExt = swModel.Extension

    'Get the annotation views
    swAnnViews = swModelExt.AnnotationViews
    For i = 0 To swModelExt.AnnotationViewCount - 1
        Set swAnnView = swAnnViews(i)
    Next
    Debug.Print "Number of annotation views = " & swModelExt.AnnotationViewCount
    Debug.Print ""

    'Iterate through the annotation views and annotations
    'Print the name of each annotation view and number of annotations
    'in that annotation view
    'Print whether each annotation in that annotation view
    'can be shown in the other annotation views and in
    'multiple annotation views
    Debug.Print " Name of annotation view and number of annotations in the annotation view "
    Debug.Print ""
    For i = 0 To swModelExt.AnnotationViewCount - 1
        Set swAnnView = swAnnViews(i)
        swAnnView.Activate
        annotations = swAnnView.GetAnnotations2(False, False)
        Set swFeat = swAnnView
        nbrAnnotations = swAnnView.AnnotationCount
        If nbrAnnotations > 0 Then
            Debug.Print ""
        End If
        Debug.Print "          " & swFeat.Name & " = " & nbrAnnotations
        If Not IsEmpty(annotations) Then
            j = 0
            For j = 0 To nbrAnnotations - 1
                Debug.Print "          Can show annotation " & j + 1 & " in these annotation views: "
                Set swAnn = annotations(j)
                For k = 0 To swModelExt.AnnotationViewCount - 1
                    Set swAnnView = swAnnViews(k)
                    Set swFeat = swAnnView
                    Debug.Print "          " & swFeat.Name & " = " & swAnn.CanShowInAnnotationView(swFeat.Name)
                Next k
                Debug.Print "          multiple = " & swAnn.CanShowInMultipleAnnotationViews()
                Debug.Print ""
            Next j
        End If
    Next i
End Sub
```

Get Where Annotations Can Be Shown Example (VB.NET)

This example shows how to get the names of the annotation views where an annotation can be shown.

```

'-----
' Preconditions:
' 1. Open a part document with one or more annotations
'    in one or more annotation views.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets the annotation views.
' 2. Iterates through the annotation views and annotations.
' 3. Prints the name of each annotation view and number of
'    annotations in that annotation view.
' 4. Prints whether each annotation in that annotation view
'    can be shown in the other annotation views and in
'    multiple annotation views.
' 5. Examine the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelExt As ModelDocExtension
        Dim swAnnViews() As Object
        Dim annotations() As Object
        Dim swAnnView As AnnotationView
        Dim swAnn As Annotation
        Dim swFeat As Feature
        Dim i As Integer
        Dim j As Integer
        Dim k As Integer
        Dim nbrAnnotations As Integer

        swModel = swApp.ActiveDoc
        swModelExt = swModel.Extension

        'Get the annotation views
        swAnnViews = swModelExt.AnnotationViews
        For i = 0 To swModelExt.AnnotationViewCount - 1
            swAnnView = swAnnViews(i)
        Next
        Debug.Print("Number of annotation views = " & swModelExt.AnnotationViewCount)
        Debug.Print("")

        'Iterate through the annotation views and annotations
        'Print the name of each annotation view and number of annotations
        'in that annotation view
        'Print whether each annotation in that annotation view
        'can be shown in the other annotation views and in
        'multiple annotation views
        Debug.Print(" Name of annotation view and number of annotations in the annotation view ")
        Debug.Print("")
        For i = 0 To swModelExt.AnnotationViewCount - 1
            swAnnView = swAnnViews(i)
            swAnnView.Activate()
            annotations = swAnnView.GetAnnotations2(False, False)
            swFeat = swAnnView
            nbrAnnotations = swAnnView.AnnotationCount
            If nbrAnnotations > 0 Then
                Debug.Print("")
            End If
            Debug.Print(" " & swFeat.Name & " = " & nbrAnnotations)
            If Not annotations Is Nothing Then
                j = 0
                For j = 0 To nbrAnnotations - 1
                    Debug.Print(" Can show annotation " & j + 1 & " in these annotation views: ")
                    swAnn = annotations(j)
                    For k = 0 To swModelExt.AnnotationViewCount - 1
                        swAnnView = swAnnViews(k)
                        swFeat = swAnnView
                        Debug.Print(" " & swFeat.Name & " = " & swAnn.CanShowInAnnotationView(swFeat.Name))
                    Next k
                    Debug.Print(" multiple = " & swAnn.CanShowInMultipleAnnotationViews())
                    Debug.Print("")
                Next j
            End If
        Next i

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get Whether Components Are Loaded Example (C#)

This example gets whether the components in an assembly document are loaded.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly document exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Loads the Magnet-1 component.
// 2. Examine the Immediate window.
//
// NOTE: Because this assembly document is used elsewhere,
// do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace IsLoadedComponent2CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel;
            DocumentSpecification swDocSpecification;
            string[] sComponents = new string[1];
            object[] Components;
            Component2 swComponent = default(Component2);
            string sName;
            AssemblyDoc swAssembly;
            int longstatus;
            int longwarnings;
            int i;
            ConfigurationManager swConfigMgr;
            Configuration swConfig;

            // Selectively open speaker.sldasm
            // Load only Magnet-1
            swDocSpecification = (DocumentSpecification)swApp.GetOpenDocSpec("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\pdmworks\\speaker.sldasm");
            sComponents[0] = "Magnet-1@speaker";
            Components = (object[])sComponents;
            swDocSpecification.ComponentList = Components;
            swDocSpecification.Selective = true;
            sName = swDocSpecification.FileName;
            swDocSpecification.DocumentType = (int)swDocumentTypes_e.swDocASSEMBLY;
            swDocSpecification.DisplayState = "Default_Display State-1";
            swDocSpecification.UseLightWeightDefault = true;
            swDocSpecification.LightWeight = true;
            swDocSpecification.Silent = true;
            swDocSpecification.IgnoreHiddenComponents = true;
            swModel = (ModelDoc2)swApp.OpenDoc7(swDocSpecification);
            longstatus = swDocSpecification.Error;
            longwarnings = swDocSpecification.Warning;

            // Get whether the components in the
            // assembly document are loaded and suppressed; only
            // Magnet-1 should be loaded and not suppressed
            swAssembly = (AssemblyDoc)swModel;
            swConfigMgr = (ConfigurationManager)swModel.ConfigurationManager;
```

```

swConfig = (Configuration)swConfigMgr.ActiveConfiguration;
Components = (object[])swAssembly.GetComponents(true);
for (i = 0; i < Components.Length; i++)
{
    swComponent = (Component2)Components[i];
    if ((swComponent.IsLoaded()))
    {
        Debug.Print("Component: " + swComponent.Name + " is loaded.");
    }
    else
    {
        Debug.Print("Component: " + swComponent.Name + " is not loaded.");
    }

    Debug.Print (" Suppressed: " + swConfig.GetComponentSuppressionState(swComponent.Name));
    Debug.Print ("");
}
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Get Whether Components Are Loaded Example (VBA)

This example gets whether the components in an assembly document are loaded.

```

'-----
' Preconditions:
' 1. Verify that the specified assembly document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Loads the Magnet-1 component.
' 2. Examine the Immediate window.
'
' NOTE: Because this assembly document is used elsewhere,
' do not save changes.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDocSpecification As SldWorks.DocumentSpecification
Dim sComponents(0) As String
Dim Components As Variant
Dim swComponent As SldWorks.Component2
Dim sName As String
Dim swAssembly As SldWorks.AssemblyDoc
Dim longstatus As Long, longwarnings As Long
Dim i As Long
Dim swConfigMgr As SldWorks.ConfigurationManager
Dim swConfig As SldWorks.Configuration

Sub main()

    Set swApp = Application.SldWorks

    ' Selectively open speaker.sldasm
    ' Load only the Magnet-1 component
    Set swDocSpecification = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS\2018\samples\tutorial\pdmworks\speaker.sldasm")
    sComponents(0) = "Magnet-1@speaker"
    Components = sComponents
    swDocSpecification.ComponentList = Components
    swDocSpecification.Selective = True
    sName = swDocSpecification.FileName
    swDocSpecification.DocumentType = swDocASSEMBLY
    swDocSpecification.DisplayState = "Default_Display State-1"
    swDocSpecification.UseLightWeightDefault = True
    swDocSpecification.LightWeight = True
    swDocSpecification.Silent = True
    swDocSpecification.IgnoreHiddenComponents = True
    Set swModel = swApp.OpenDoc7(swDocSpecification)
    longstatus = swDocSpecification.Error
    longwarnings = swDocSpecification.Warning

    ' Get whether the components in the
    ' assembly document are loaded and suppressed; only
    ' Magnet-1 should be loaded and not suppressed
    Set swAssembly = swModel
    Set swConfigMgr = swModel.ConfigurationManager
    Set swConfig = swConfigMgr.ActiveConfiguration
    Components = swAssembly.GetComponents(True)
    For i = 0 To UBound(Components)
        Set swComponent = Components(i)
        If (swComponent.IsLoaded) Then

```

```
        Debug.Print "Component: " & swComponent.Name & " is loaded."
    Else
        Debug.Print "Component: " & swComponent.Name & " is not loaded. "
    End If

    Debug.Print " Suppressed: " & swConfig.GetComponentSuppressionState(swComponent.Name)
    Debug.Print ""

Next
End Sub
```

Get Whether Components Are Loaded Example (VB.NET)

This example gets whether the components in an assembly document are loaded.

```
'-----
' Preconditions:
' 1. Verify that the specified assembly document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Loads the Magnet-1 component.
' 2. Examine the Immediate window.
' NOTE: Because this assembly document is used elsewhere,
' do not save changes.
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim swModel As ModelDoc2
```

```
        Dim swDocSpecification As DocumentSpecification
```

```
        Dim sComponents(0) As String
```

```
        Dim Components As Object
```

```
        Dim swComponent As Component2
```

```
        Dim sName As String
```

```
        Dim swAssembly As AssemblyDoc
```

```
        Dim longstatus As Integer
```

```
        Dim longwarnings As Integer
```

```
        Dim i As Integer
```

```
        Dim swConfigMgr As ConfigurationManager
```

```
        Dim swConfig As Configuration
```

```
        ' Selectively open speaker.sldasm assembly
```

```
        ' Load only the Magnet-1 component
```

```
        swDocSpecification = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\pdmworks\speaker.sldasm")
```

```
        sComponents(0) = "Magnet-1@speaker"
```

```
        Components = sComponents
```

```
        swDocSpecification.ComponentList = Components
```

```
        swDocSpecification.Selective = True
```

```
        sName = swDocSpecification.FileName
```

```
        swDocSpecification.DocumentType = swDocumentTypes_e.swDocASSEMBLY
```

```
        swDocSpecification.DisplayState = "Default_Display State-1"
```

```
        swDocSpecification.UseLightWeightDefault = True
```

```
        swDocSpecification.LightWeight = True
```

```
        swDocSpecification.Silent = True
```

```
        swDocSpecification.IgnoreHiddenComponents = True
```

```
        swModel = swApp.OpenDoc7(swDocSpecification)
```

```
        longstatus = swDocSpecification.Error
```

```
        longwarnings = swDocSpecification.Warning
```

```
        ' Get whether the components in the
```

```
        ' assembly document are loaded and suppressed; only
```

```
        ' Magnet-1 should be loaded and not suppressed
```

```
        swAssembly = swModel
```

```
        swConfigMgr = swModel.ConfigurationManager
```

```
        swConfig = swConfigMgr.ActiveConfiguration
```

```
        Components = swAssembly.GetComponents(True)
```

```

For i = 0 To UBound(Components)
    swComponent = Components(i)
    If (swComponent.IsLoaded) Then
        Debug.Print("Component: " & swComponent.Name & " is loaded.")
    Else
        Debug.Print("Component: " & swComponent.Name & " is not loaded.")
    End If

    Debug.Print("  Suppressed: " & swConfig.GetComponentSuppressionState(swComponent.Name))
    Debug.Print("")

Next
End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Get Whether Display Dimension is a Hole Callout Example (VBA)

This example shows how to determine if a display dimension is a hole callout.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\costing\machined_part.sldprt.
' 2. Click File > Make Drawing from Part > OK, drag-and-drop
'    (A)Top in the drawing, and click OK in the PropertyManager page.
' 3. Click Insert > Annotations > Hole Callout, click any
'    hole in the model, drag-and-drop the hole
'    callout in an empty space in the drawing, and click OK
'    in the PropertyManager page.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Selects RD1@DrawingView1.
' 2. Gets whether RD1@DrawingView1 is a hole callout and
'    gets its text.
' 3. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swDisplayDimension As SldWorks.DisplayDimension
Dim boolstatus As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swModelDocExt = swModel.Extension

    ' Select hole callout and get its text
    boolstatus = swModelDocExt.SelectByID2("RD1@Drawing View1", "DIMENSION", 0.2385689138462, 0.1081692215385, 0, False, 0, Nothing, 0)
    Set swDisplayDimension = swSelMgr.GetSelectedObject6(1, -1)
    Debug.Print "Is a hole callout? " & swDisplayDimension.IsHoleCallout
    Debug.Print " Callout portion above text = " & swDisplayDimension.GetText(swDimensionTextParts_e.swDimensionTextCalloutAbove)
    Debug.Print " Callout portion below text = " & swDisplayDimension.GetText(swDimensionTextParts_e.swDimensionTextCalloutBelow)
    Debug.Print " Prefix of callout = " & swDisplayDimension.GetText(swDimensionTextParts_e.swDimensionTextPrefix)
    Debug.Print " Suffix of callout = " & swDisplayDimension.GetText(swDimensionTextParts_e.swDimensionTextSuffix)

    swModel.ClearSelection2 True

End Sub

```

Get Whether Draft Edges Scaled in Printed Drawing Example (C#)

This example shows how to get whether draft edges are scaled when printing a drawing in high quality.

```
//-----
// Preconditions:
// 1. Verify that the specified drawing exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified drawing.
// 2. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace ScaleDraftEdgesCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            DrawingDoc swDrawing = default(DrawingDoc);
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            PageSetup swPageSetup = default(PageSetup);
            string fileName = null;
            int errors = 0;
            int warnings = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\advdrawings\\foodprocessor.slddrw";
            swDrawing = (DrawingDoc)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref
errors, ref warnings);
            swModel = (ModelDoc2)swDrawing;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swPageSetup = (PageSetup)swModelDocExt.AppPageSetup;
            if (swPageSetup.HighQuality)
            {
                Debug.Print("Drawing set to print in high quality. Are draft edges scaled? " + swPageSetup.ScaleDraftEdges);
            }
            else
            {
                Debug.Print("Drawing not set to print in high quality. Can only scale draft edges when drawing set to print in high quality.");
            }
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get Whether Draft Edges Scaled in Printed Drawing Example (VBA)

This example shows how to get whether draft edges are scaled when printing a drawing in high quality.

```
'-----  
' Preconditions:  
' 1. Verify that the specified drawing exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the drawing.  
' 2. Examine the Immediate window.  
'-----  
Dim swApp As SldWorks.SldWorks  
Dim swDrawing As SldWorks.DrawingDoc  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swPageSetup As SldWorks.PageSetup  
Dim fileName As String  
Dim errors As Long  
Dim warnings As Long  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\foodprocessor.slddrw"  
    Set swDrawing = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
    Set swModel = swDrawing  
    Set swModelDocExt = swModel.Extension  
    Set swPageSetup = swModelDocExt.AppPageSetup  
    If swPageSetup.HighQuality Then  
        Debug.Print "Drawing set to print in high quality. Are draft edges scaled? " & swPageSetup.ScaleDraftEdges  
    Else  
        Debug.Print "Drawing not set to print in high quality. Can only scale draft edges when drawing set to print in high quality."  
    End If  
  
End Sub
```

Get Whether Draft Edges Scaled in Printed Drawing Example (VB.NET)

This example shows how to get whether draft edges are scaled when printing a drawing in high quality.

```
-----
| Preconditions:
| 1. Verify that the specified drawing exists.
| 2. Open the Immediate window.
|
| Postconditions:
| 1. Opens the drawing.
| 2. Examine the Immediate window.
|-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swDrawing As DrawingDoc
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swPageSetup As PageSetup
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\foodprocessor.slddrw"
        swDrawing = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModel = swDrawing
        swModelDocExt = swModel.Extension
        swPageSetup = swModelDocExt.AppPageSetup
        If swPageSetup.HighQuality Then
            Debug.Print("Drawing set to print in high quality. Are draft edges scaled? " & swPageSetup.ScaleDraftEdges)
        Else
            Debug.Print("Drawing not set to print in high quality. Can only scale draft edges when drawing set to print in high quality.")
        End If

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Get Whether Feature is Frozen Example (C#)

This example shows how to get whether a feature is frozen.

```
//-----
// Preconditions:
// 1. Open a part or assembly.
// 2. Select a frozen feature in the FeatureManager design tree.
//
// Postconditions: Inspect the Immediate Window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace FeatureIsFrozen_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeat = default(Feature);
            bool bRet = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);

            bRet = swFeat.IsFrozen();
            Debug.Print("Feature is frozen? " + bRet);
        }

        public SldWorks swApp;
    }
}
```

Get Whether Feature is Frozen Example (VBA)

This example shows how to get whether a feature is frozen.

```
'-----  
' Preconditions:  
' 1. Open a part or assembly.  
' 2. Select a frozen feature in the FeatureManager design tree.  
'-----  
' Postconditions: Inspect the Immediate Window.  
'-----
```

Option Explicit

Sub main()

```
    Dim swApp           As SldWorks.SldWorks  
    Dim swModel         As SldWorks.ModelDoc2  
    Dim swSelMgr        As SldWorks.SelectionMgr  
    Dim swFeat          As SldWorks.Feature  
    Dim bRet            As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)  
  
    bRet = swFeat.IsFrozen  
    Debug.Print "Feature is frozen? " & bRet
```

End Sub

Get Whether Feature is Frozen Example (VB.NET)

This example shows how to get whether a feature is frozen.

```
'-----  
' Preconditions:  
' 1. Open a part or assembly.  
' 2. Select a frozen feature in the FeatureManager design tree.  
'-----  
' Postconditions: Inspect the Immediate Window.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```
        Dim swModel As ModelDoc2  
        Dim swSelMgr As SelectionMgr  
        Dim swFeat As Feature  
        Dim bRet As Boolean
```

```
        swModel = swApp.ActiveDoc  
        swSelMgr = swModel.SelectionManager  
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
```

```
        bRet = swFeat.IsFrozen  
        Debug.Print("Feature is frozen? " & bRet)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Get Whether HLR/HLV Edges Are Displayed Faceted Example (VBA)

This example gets whether HLR or HLV edges (Hidden Lines Removed or Hidden Lines Visible) in the selected view are displayed faceted or not.

```
'-----
' Preconditions:
' 1. Open a drawing and select a drawing view.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets the pathname of the drawing, the selected
'    view name, and whether HLR/HLV edges are
'    displayed faceted in the drawing view.
' 2. Examine the Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swView As SldWorks.View
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swView = swSelMgr.GetSelectedObject6(1, -1)

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " " & swView.Name & " [" & swView.GetDisplayMode2 & "]"
    Debug.Print "    HLR/HLV edges displayed faceted in this drawing view = " & swView.GetFacettedHlrDisplay

End Sub
```

Get Whether Linear Dimension Is Foreshortened Example (C#)

This example shows how to get whether a linear dimension is foreshortened.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\chair.slddrw.
// 2. Click Tools > Options > Document Properties > Dimensions > Linear.
// 3. Click ANSI in Base linear dimension standard.
// 4. Verify that the following check box and option are selected in
// Foreshortened:
// * Automatic
// * Zigzag
// 5. Click OK.
// 6. Dimension an outside linear edge.
//
// Postconditions:
// 1. Gets whether the dimension is foreshortened.
// 2. Examine the Immediate window and drawing.
//
// NOTES:
// * Foreshortened dimensions are only valid for
//   linear dimensions and only when the detailing standard
//   is ANSI.
// * Because the part and drawing are used elsewhere, do not
//   save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            DisplayDimension swDisplayDimension = default(DisplayDimension);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swDisplayDimension = (DisplayDimension)swSelectionMgr.GetSelectedObject6(1, -1);
            Debug.Print("Foreshortened dimension? " + swDisplayDimension.Foreshortened);

            swModel.ClearSelection2(true);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get Whether Linear Dimension Is Foreshortened Example (VBA)

This example shows how to get whether a linear dimension is foreshortened.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\chair.slddrw.
' 2. Click Tools > Options > Document Properties > Dimensions > Linear.
' 3. Click ANSI in Base linear dimension standard.
' 4. Verify that the following check box and option are selected in
'    Foreshortened:
'      * Automatic
'      * Zigzag
' 5. Click OK.
' 6. Dimension an outside linear edge.
'
' Postconditions:
' 1. Gets whether the dimension is foreshortened.
' 2. Examine the Immediate window and drawing.
'
' NOTES:
' * Foreshortened dimensions are only valid for
'   linear dimensions and only when the detailing standard
'   is ANSI.
' * Because the part and drawing are used elsewhere, do not
'   save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swDisplayDimension As SldWorks.DisplayDimension

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelectionMgr = swModel.SelectionManager
    Set swDisplayDimension = swSelectionMgr.GetSelectedObject6(1, -1)
    Debug.Print "Foreshortened dimension? " & swDisplayDimension.Foreshortened

    swModel.ClearSelection2 True

End Sub
```

Get Whether Linear Dimension Is Foreshortened Example (VB.NET)

This example shows how to get whether a linear dimension is foreshortened.

```

-----
Preconditions:
1. Open public_documents\samples\tutorial\api\chair.slddrw.
2. Click Tools > Options > Document Properties > Dimensions > Linear.
3. Click ANSI in Base linear dimension standard.
4. Verify that the following check box and option are selected in
   Foreshortened:
   * Automatic
   * Zigzag
5. Click OK.
6. Dimension an outside linear edge.

Postconditions:
1. Gets whether the dimension is foreshortened.
2. Examine the Immediate window and drawing.

NOTES:
* Foreshortened dimensions are only valid for
  linear dimensions and only when the detailing standard
  is ANSI.
* Because the part and drawing are used elsewhere, do not
  save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swSelectionMgr As SelectionMgr
        Dim swDisplayDimension As DisplayDimension

        swModel = swApp.ActiveDoc
        swSelectionMgr = swModel.SelectionManager
        swDisplayDimension = swSelectionMgr.GetSelectedObject6(1, -1)
        Debug.Print("Foreshortened dimension? " & swDisplayDimension.Foreshortened)

        swModel.ClearSelection2(True)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get Whether Note Contains Rich Embedded Text Example (VBA)

This example shows how to find out if a note contains rich embedded text.

```
'-----
' Preconditions:
' 1. Open a model document containing a
'    multi-string note.
' 2. Italicize one of the strings in the note.
' 3. Select the note.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Gets:
'    a. selected note.
'    b. whether the note contains rich-embedded
'       text.
' 2. Examine the Immediate window.
' 3. Uncomment the executable commented code statements.
' 4. Run the macro again.
' 5. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModelDoc As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swNote As SldWorks.Note
Dim swTextFormat As SldWorks.TextFormat
Dim swAnnotation As SldWorks.Annotation
Dim boolstatus As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModelDoc = swApp.ActiveDoc
    Set swSelMgr = swModelDoc.SelectionManager
    Set swNote = swSelMgr.GetSelectedObject6(1, 0)
    'Set swAnnotation = swNote.GetAnnotation
    'Set swTextFormat = swAnnotation.GetTextFormat(0)
    'swTextFormat.Bold = True
    ' IAnnotation::SetTextFormat should return false because
    ' note contains rich-embedded text
    'boolstatus = swAnnotation.SetTextFormat(0, False, swTextFormat)
    'Debug.Print "Text format set? " & boolstatus
    boolstatus = swNote.HasMultipleFonts
    Debug.Print "Does selected note have multiple fonts? " & boolstatus

End Sub
```

Get Whether Sketch Segment is Trimmed or Not Example (VBA)

This example shows how to determine if a sketch segment is trimmed or not.

```
'-----
' Preconditions:
' 1. Open a part that contains a 3D sketch containing
'    a sketch segment (i.e., line, arc, ellipse,
'    spline, or parabola).
' 2. Select the 3D sketch.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets the sketch segment and its ID and type
'    and whether the sketch segment is construction
'    geometry and trimmed.
' 2. Examine the Immediate window.
'-----
Option Explicit

Public Enum swSkSegments_e
    swSketchLINE = 0
    swSketchARC = 1
    swSketchELLIPSE = 2
    swSketchSPLINE = 3
    swSketchTEXT = 4
    swSketchPARABOLA = 5
End Enum

Sub main()

    Dim sSkSegmentsName(5) As String
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swSkSeg As SldWorks.SketchSegment
    Dim swCurve As SldWorks.Curve
    Dim swSkFeat As SldWorks.Feature
    Dim swSketch As SldWorks.Sketch
    Dim vID As Variant
    Dim bRet As Boolean

    sSkSegmentsName(swSketchLINE) = "swSketchLINE"
    sSkSegmentsName(swSketchARC) = "swSketchARC"
    sSkSegmentsName(swSketchELLIPSE) = "swSketchELLIPSE"
    sSkSegmentsName(swSketchSPLINE) = "swSketchSPLINE"
    sSkSegmentsName(swSketchTEXT) = "swSketchTEXT"
    sSkSegmentsName(swSketchPARABOLA) = "swSketchPARABOLA"

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swSkSeg = swSelMgr.GetSelectedObject5(1)
    Set swCurve = swSkSeg.GetCurve
    Set swSkFeat = swSkSeg.GetSketch
    Set swSketch = swSkFeat.GetSpecificFeature

    vID = swSkSeg.GetID

    Debug.Print "Feature = " & swSkFeat.Name & " [" & swSketch.Is3D & "]"
    Debug.Print "    Sketch Segment"
    Debug.Print "        ID = [" & vID(0) & ", " & vID(1) & "]"
    Debug.Print "            Type = " & sSkSegmentsName(swSkSeg.GetType)
    Debug.Print "            Construction geometry = " & swSkSeg.ConstructionGeometry
    Debug.Print "            Is trimmed = " & swCurve.IsTrimmedCurve

End Sub
```

Get Hole Standards Data Example (C#)

This example shows how to retrieve hole standards data from the Hole Wizard database.

```
//-----
// Preconditions:
// 1. Open SOLIDWORKS Design.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Retrieves row 1 data from all three Binding Head Screw fastener tables
//    of the ANSI Inch hole standard.
// 2. Inspect the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace GetHoleStandards_CSharp
{
    partial class SolidWorksMacro
    {
        private HoleStandardsData holeStandards;
        private object fastenerData;
        private HoleDataTable fastenerDataObject;
        private bool ret;
        private object indexes;
        private object standards;
        private object[] standardsArray;
        private object fastenerIDs;
        private int[] fastenerIDsArray;
        private object fastenerNames;
        private object[] fastenerNamesArray;
        private object fastenerTableTypeIDs;
        private int[] fastenerTableTypeIDsArray;
        private object columnNames;
        private object[] columnNamesArray;
        private string cellData;
        private int i;
        private int j;

        public void Main()
        {
            holeStandards = (HoleStandardsData)swApp.GetHoleStandardsData((int)swWzdGeneralHoleTypes_e.swWzdCounterBore);
            ret = holeStandards.GetHoleStandards(out indexes, out standards);
            standardsArray = (object[])standards;
            ret = holeStandards.GetFastenerTypes((string)standardsArray[0], out fastenerIDs, out fastenerNames);
            fastenerIDsArray = (int[])fastenerIDs;
            fastenerNamesArray = (object[])fastenerNames;
            ret = holeStandards.GetFastenerTableTypes((string)standardsArray[0], (int)fastenerIDsArray[0], out fastenerTableTypeIDs);
            fastenerTableTypeIDsArray = (int[])fastenerTableTypeIDs;
            for (j = 0; j <= 2; j++)
            {
                ret = holeStandards.GetFastenerTable((string)standardsArray[0], (int)fastenerIDsArray[0], (int)fastenerTableTypeIDs);
                fastenerDataObject = (HoleDataTable)fastenerData;
                Debug.Print("");
                if (j == 0)
                {
                    Debug.Print("Hole Wizard standard: " + fastenerDataObject.StandardName);
                    Debug.Print("Fastener ID: " + fastenerDataObject.FastenerID);
                    Debug.Print("Fastener: " + fastenerNamesArray[0]);
                }

                Debug.Print("Fastener table type ID as defined in swFastenerTableTypes_e: " + fastenerDataObject.TableTypeID);

                ret = fastenerDataObject.GetColumnNames(out columnNames);
                columnNamesArray = (object[])columnNames;
                if (j == 0)
                {
                    Debug.Print("Size data in Row #1");
                }
                else if (j == 1)
                {
                    Debug.Print("Thread data in Row #1");
                }
                else if (j == 2)
                {
                    Debug.Print("Screw clearances data in Row #1");
                }

                for (i = 0; i <= columnNamesArray.GetUpperBound(0); i++)
                {

```

```

        ret = fastenerDataObject.GetCellData((string)columnNamesArray[i], 0, out cellData);
        Debug.Print(columnNamesArray[i] + ": " + cellData);
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}

}

```

Get Hole Standards Data Example (VBA)

This example shows how to retrieve hole standards data from the Hole Wizard database.

```

'-----
' Preconditions:
' 1. Open SOLIDWORKS Design.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Retrieves row 1 data from all three Binding Head Screw fastener tables
'   of the ANSI Inch hole standard.
' 2. Inspect the Immediate window.
'-----

Dim swApp As SldWorks.SldWorks
Dim holeStandards As SldWorks.HoleStandardsData
Dim fastenerData As SldWorks.HoleDataTable
Dim ret As Boolean
Dim indexes As Variant
Dim standards As Variant
Dim fastenerIDs As Variant
Dim fastenerNames As Variant
Dim fastenerTableTypeIDs As Variant
Dim columnNames As Variant
Dim cellData As String
Dim i As Long
Dim j As Long
Option Explicit
Sub main()

    Set swApp = Application.SldWorks
    Set holeStandards = swApp.GetHoleStandardsData(swWzdCounterBore)
    ret = holeStandards.GetHoleStandards(indexes, standards)
    ret = holeStandards.GetFastenerTypes(standards(0), fastenerIDs, fastenerNames)
    ret = holeStandards.GetFastenerTableTypes(standards(0), fastenerIDs(0), fastenerTableTypeIDs)
    For j = 0 To 2
        ret = holeStandards.GetFastenerTable(standards(0), fastenerIDs(0), fastenerTableTypeIDs(j), fastenerData)
        Debug.Print ""
        If j = 0 Then
            Debug.Print "Hole Wizard standard: " & fastenerData.StandardName
            Debug.Print "Fastener ID: " & fastenerData.FastenerID
            Debug.Print "Fastener: " & fastenerNames(0)
        End If

        Debug.Print "Fastener table type ID as defined in swFastenerTableTypes_e: " & fastenerData.TableTypeID

        ret = fastenerData.GetColumnNames(columnNames)
        If j = 0 Then
            Debug.Print "Size data in Row #1"
        ElseIf j = 1 Then
            Debug.Print "Thread data in Row #1"
        ElseIf j = 2 Then
            Debug.Print "Screw clearances data in Row #1"
        End If

        For i = 0 To UBound(columnNames)
            ret = fastenerData.GetCellData(columnNames(i), 0, cellData)
            Debug.Print columnNames(i) & ": " & cellData
        Next i
    Next j
End Sub

```

Get x,y,z Locations of Points in Sketch Example (VBA)

This example shows how to get the x, y, and z locations of the points in the selected sketch.

```
'-----
' Preconditions:
' 1. Open a part.
' 2. Select a sketch.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets the IDs of the sketch points in the
'    sketch and their model and sketch point
'    x, y, and z locations.
' 2. Examine the Immediate window.
'-----
Option Explicit

Sub ProcessSketchPoint(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSkPt As SldWorks.SketchPoint)
    Dim vID As Variant
    Dim swSketch As SldWorks.Sketch
    Dim swMathUtil As SldWorks.MathUtility
    Dim swXform As SldWorks.MathTransform
    Dim nPt(2) As Double
    Dim vPt As Variant
    Dim swMathPt As SldWorks.MathPoint

    Set swSketch = swSkPt.GetSketch
    vID = swSkPt.GetID
    Debug.Print "    ID = [" & vID(0) & ", " & vID(1) & "]"
    If swSketch.Is3D Then
        ' Point is already in model space
        Debug.Print "    Point (model)    = (" & swSkPt.X * 1000# & ", " & swSkPt.Y * 1000# & ", " & swSkPt.Z * 1000# & ") mm"
    Else
        nPt(0) = swSkPt.X: nPt(1) = swSkPt.Y: nPt(2) = swSkPt.Z
        vPt = nPt
        Set swXform = swSketch.ModelToSketchTransform
        Set swXform = swXform.Inverse
        Set swMathUtil = swApp.GetMathUtility
        Set swMathPt = swMathUtil.CreatePoint((vPt))
        Set swMathPt = swMathPt.MultiplyTransform(swXform)
        Debug.Print "    Point (model)    = (" & swMathPt.ArrayData(0) * 1000# & ", " & swMathPt.ArrayData(1) * 1000# & ", " & swMathPt.ArrayData(2) *
1000# & ") mm"
        Debug.Print "    Point (sketch)   = (" & swSkPt.X * 1000# & ", " & swSkPt.Y * 1000# & ", " & swSkPt.Z * 1000# & ") mm"
    End If
End Sub

Sub main()
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As SldWorks.Feature
    Dim swSketch As SldWorks.Sketch
    Dim vSkPtArr As Variant
    Dim vSkPt As Variant
    Dim swSkPt As SldWorks.SketchPoint
    Dim i As Long
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject6(1, 0)
    Set swSketch = swFeat.GetSpecificFeature2
    Debug.Print "Feature = " & swFeat.Name
    Debug.Print " Sketch Points:"

    vSkPtArr = swSketch.GetSketchPoints2: If IsEmpty(vSkPtArr) Then Exit Sub
    For Each vSkPt In vSkPtArr
        Set swSkPt = vSkPt
        ProcessSketchPoint swApp, swModel, swSkPt
    Next vSkPt
End Sub
```

Group and Ungroup Components Example (C#)

This example shows how to:

- group the same components in the same configuration in an assembly into a folder in the FeatureManager design tree.
- ungroup the grouped components.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\advdrawings\98food_processor.sldasm.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Groups the rubber feet components into rubber feet(Default)(5).
// 2. Examine the Immediate window and FeatureManager design tree.
// 3. Press F5.
// 4. Ungroups rubber feet(Default)(5).
// 5. Examine the Immediate window and FeatureManager design tree.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            FeatureManager swFeatureManager = default(FeatureManager);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            AssemblyDoc swAssemblyDoc = default(AssemblyDoc);
            bool status = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;

            //Group rubber feet components into rubber feet(Default)(5)
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swFeatureManager.GroupComponentInstances = true;
            Debug.Print("Component instances grouped? " + swFeatureManager.GroupComponentInstances);

            System.Diagnostics.Debugger.Break();
            //Press F5

            //Ungroup rubber feet(Default)(5)
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("rubber feet(Default)(5)", "FTRFOLDER", 0, 0, 0, false, 0, null, 0);
            swAssemblyDoc = (AssemblyDoc)swModel;
            status = swAssemblyDoc.UngroupComponents();
            Debug.Print("Grouped components ungrouped? " + status);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Group and Ungroup Components Example (VBA)

This example shows how to:

- group the same components in the same configuration in an assembly into a folder in the FeatureManager design tree.
- ungroup the grouped components.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\advdrawings\98food processor.sldasm.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Groups the rubber feet components into rubber feet(Default)(5).
' 2. Examine the Immediate window and FeatureManager design tree.
' 3. Press F5.
' 4. Ungroups rubber feet(Default)(5).
' 5. Examine the Immediate window and FeatureManager design tree.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swFeatureManager As SldWorks.FeatureManager
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swAssemblyDoc As SldWorks.AssemblyDoc
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    'Group rubber feet components into rubber feet(Default)(5)
    Set swFeatureManager = swModel.FeatureManager
    swFeatureManager.GroupComponentInstances = True
    Debug.Print "Component instances grouped? " & swFeatureManager.GroupComponentInstances

    Stop
    'Press F5

    'Ungroup rubber feet(Default)(5)
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("rubber feet(Default)(5)", "FTRFOLDER", 0, 0, 0, False, 0, Nothing, 0)
    Set swAssemblyDoc = swModel
    status = swAssemblyDoc.UngroupComponents
    Debug.Print "Grouped components ungrouped? " & status

End Sub

```

Group and Ungroup Components Example (VB.NET)

This example shows how to:

- group the same components in the same configuration in an assembly into a folder in the FeatureManager design tree.
- ungroup the grouped components.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\advdrawings\98food_processor.sldasm.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Groups the rubber feet components into rubber feet(Default)(5).
' 2. Examine the Immediate window and FeatureManager design tree.
' 3. Press F5.
' 4. Ungroups rubber feet(Default)(5).
' 5. Examine the Immediate window and FeatureManager design tree.
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swFeatureManager As FeatureManager
        Dim swModelDocExt As ModelDocExtension
        Dim swAssemblyDoc As AssemblyDoc
        Dim status As Boolean

        swModel = swApp.ActiveDoc

        'Group rubber feet components into rubber feet(Default)(5)
        swFeatureManager = swModel.FeatureManager
        swFeatureManager.GroupComponentInstances = True
        Debug.Print("Component instances grouped? " & swFeatureManager.GroupComponentInstances)

        Stop
        'Press F5

        'Ungroup rubber feet(Default)(5)
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("rubber feet(Default)(5)", "FTRFOLDER", 0, 0, 0, False, 0, Nothing, 0)
        swAssemblyDoc = swModel
        status = swAssemblyDoc.UngroupComponents
        Debug.Print("Grouped components ungrouped? " & status)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Hide All Edges in Drawing View Example (VBA)

This example shows how to hide all of the visible edges in the root component in a drawing view.

```
'-----
' Preconditions:
' 1. Verify that the drawing document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing document.
' 2. Gets the current sheet and Drawing View1.
' 3. Gets the visible entities in the root component in
'   Drawing View1.
' 4. Hides all of the visible edges in the root component
'   in Drawing View1.
' 5. Examine Drawing View1 and the Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDraw As SldWorks.DrawingDoc
Dim swDocSpecification As SldWorks.DocumentSpecification
Dim swSheet As SldWorks.Sheet
Dim swView As SldWorks.View
Dim swDrawingComponent As SldWorks.DrawingComponent
Dim swComponent As SldWorks.Component2
Dim swEntity As SldWorks.Entity
Dim vEdges As Variant
Dim bRet As Boolean
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks

    ' Open the drawing document
    Set swDocSpecification = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\advdrawings\foodprocessor.slddrw")
    Set swModel = swApp.ActiveDoc

    If swModel Is Nothing Then
        Set swModel = swApp.OpenDoc7(swDocSpecification)
    End If

    Set swDraw = swModel

    ' Get the current sheet
    Set swSheet = swDraw.GetCurrentSheet
    Debug.Print swSheet.GetName

    ' Select Drawing View1
    bRet = swModel.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0#, 0#, 0#, True, 0, Nothing, swSelectOptionDefault): Debug.Assert bRet
    Set swView = swModel.SelectionManager.GetSelectedObject6(1, -1)

    ' Print the drawing view name and get the component in the drawing view
    Debug.Print swView.GetName2
    Set swDrawingComponent = swView.RootDrawingComponent
    Set swComponent = swDrawingComponent.Component

    ' Get the component's visible entities in the drawing view
    vEdges = swView.GetVisibleEntities2(swComponent, swViewEntityType_Edge)

    ' Hide all of the visible edges in the drawing view
    For i = 0 To UBound(vEdges)
        Set swEntity = vEdges(i)
        swEntity.Select4 True, Nothing
        swDraw.HideEdge
    Next i

    swModel.ClearSelection2 True

End Sub
```

Hide and Show All Edges in Drawing View (VBA)

This example shows how to hide and then show all of the edges in the root component in a drawing view.

```
'-----
' Preconditions:
' 1. Verify that the drawing document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing document.
' 2. Hides and then shows all edges in the root component in
'    Drawing View1.
' 3. Examine the drawing and Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDraw As SldWorks.DrawingDoc
Dim swDocSpecification As SldWorks.DocumentSpecification
Dim swSheet As SldWorks.Sheet
Dim swView As SldWorks.View
Dim swDrawingComponent As SldWorks.DrawingComponent
Dim swComponent As SldWorks.Component2
Dim swEntity As SldWorks.Entity
Dim vEdges As Variant
Dim bRet As Boolean
Dim i As Long

Sub main()
    Set swApp = Application.SldWorks

    ' Specify the drawing to open
    Set swDocSpecification = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\advdrawings\foodprocessor.SLDDRW")
    Set swModel = swApp.ActiveDoc

    If swModel Is Nothing Then
        Set swModel = swApp.OpenDoc7(swDocSpecification)
    End If

    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel

    ' Get the current sheet
    Set swSheet = swDraw.GetCurrentSheet
    Debug.Print swSheet.GetName

    ' Select Drawing View1
    bRet = swModel.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0#, 0#, 0#, True, 0, Nothing, swSelectOptionDefault):
    Debug.Assert bRet
    Set swView = swModel.SelectionManager.GetSelectedObject6(1, -1)

    ' Print the drawing view name and get the component in the drawing view
    Debug.Print swView.GetName2
    Set swDrawingComponent = swView.RootDrawingComponent
    Set swComponent = swDrawingComponent.Component

    ' Get the component's visible entities in the drawing view
    Dim eCount As Long
    eCount = swView.GetVisibleEntityCount2(swComponent, swViewEntityType_Edge)
    vEdges = swView.GetVisibleEntities2(swComponent, swViewEntityType_Edge)
    Debug.Print "Number of edges found: " & eCount

    ' Hide all of the visible edges in the drawing view
    For i = 0 To eCount - 1
        Set swEntity = vEdges(i)
        swEntity.Select4 True, Nothing
        swDraw.HideEdge
    Next i

    ' Clear all selections
    swModel.ClearSelection2 True

    ' Show all hidden edges
    swView.HiddenEdges = vEdges
End Sub
```

Hide and Show All Edges in Drawing View (C#)

This example shows how to hide and then show all of the edges in the root component in a drawing view.

```
//-----
// Preconditions:
// 1. Verify that the specified drawing document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified drawing document.
// 2. Hides and then shows all edges in the root component in
//    Drawing View1.
// 3. Examine the drawing and Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace HideShowEdges_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        DrawingDoc swDraw;
        DocumentSpecification swDocSpecification;
        Sheet swSheet;
        View swView;
        DrawingComponent swDrawingComponent;
        Component2 swComponent;
        Entity swEntity;
        object[] vEdges;
        bool bRet;
        int i;

        public void Main()
        {
            // Specify the drawing to open
            swDocSpecification = (DocumentSpecification)swApp.GetOpenDocSpec("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\advdrawings\\foodprocessor.SLDDRW");
            swModel = (ModelDoc2)swApp.ActiveDoc;

            if (swModel == null)
            {
                swModel = swApp.OpenDoc7(swDocSpecification);
            }

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swDraw = (DrawingDoc)swModel;

            // Get the current sheet
            swSheet = (Sheet)swDraw.GetCurrentSheet();
            Debug.Print(swSheet.GetName());

            // Select Drawing View1
            bRet = swModel.Extension.SelectByID("Drawing View1", "DRAWINGVIEW", 0.0, 0.0, 0.0, true, 0, null,
(int)swSelectOption_e.swSelectOptionDefault);
            swView = (View)((SelectionMgr)swModel.SelectionManager).GetSelectedObject6(1, -1);

            // Print the drawing view name and get the component in the drawing view
            Debug.Print(swView.GetName2());
            swDrawingComponent = swView.RootDrawingComponent;
            swComponent = swDrawingComponent.Component;

            // Get the component's visible entities in the drawing view
            int eCount = 0;
            eCount = swView.GetVisibleEntityCount2(swComponent, (int)swViewEntityType_e.swViewEntityType_Edge);
            vEdges = (object[])swView.GetVisibleEntities2(swComponent, (int)swViewEntityType_e.swViewEntityType_Edge);
            Debug.Print("Number of edges found: " + eCount);

            // Hide all of the visible edges in the drawing view
            for (i = 0; i <= eCount - 1; i++)
            {
                swEntity = (Entity)vEdges[i];
                swEntity.Select4(true, null);
                swDraw.HideEdge();
            }

            // Clear all selections
            swModel.ClearSelection2(true);

            // Show all hidden edges
            swView.HiddenEdges = vEdges;
        }
    }
}
```

```
    }  
    public SldWorks swApp;  
  }  
}
```

Hide and Show All Edges in Drawing View (VB.NET)

This example shows how to hide and then show all of the edges in the root component in a drawing view.

```

'-----
' Preconditions:
' 1. Verify that the drawing document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing document.
' 2. Hides and then shows all edges in the root component in
'    Drawing View1.
' 3. Examine the drawing and Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swDraw As DrawingDoc
    Dim swDocSpecification As DocumentSpecification
    Dim swSheet As Sheet
    Dim swView As View
    Dim swDrawingComponent As DrawingComponent
    Dim swComponent As Component2
    Dim swEntity As Entity
    Dim vEdges As Object
    Dim bRet As Boolean
    Dim i As Integer

    Sub main()

        ' Specify the drawing to open
        swDocSpecification = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\advdrawings\foodprocessor.SLDDRW")
        swModel = swApp.ActiveDoc

        If swModel Is Nothing Then
            swModel = swApp.OpenDoc7(swDocSpecification)
        End If

        swModel = swApp.ActiveDoc
        swDraw = swModel

        ' Get the current sheet
        swSheet = swDraw.GetCurrentSheet
        Debug.Print(swSheet.GetName)

        ' Select Drawing View1
        bRet = swModel.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0.0#, 0.0#, 0.0#, True, 0, Nothing,
swSelectOption_e.swSelectOptionDefault)
        swView = swModel.SelectionManager.GetSelectedObject6(1, -1)

        ' Print the drawing view name and get the component in the drawing view
        Debug.Print(swView.GetName2)
        swDrawingComponent = swView.RootDrawingComponent
        swComponent = swDrawingComponent.Component

        ' Get the component's visible entities in the drawing view
        Dim eCount As Integer
        eCount = swView.GetVisibleEntityCount2(swComponent, swViewEntityType_e.swViewEntityType_Edge)
        vEdges = swView.GetVisibleEntities2(swComponent, swViewEntityType_e.swViewEntityType_Edge)
        Debug.Print("Number of edges found: " & eCount)

        ' Hide all of the visible edges in the drawing view
        For i = 0 To eCount - 1
            swEntity = vEdges(i)
            swEntity.Select4(True, Nothing)
            swDraw.HideEdge()
        Next i

        ' Clear all selections
        swModel.ClearSelection2(True)

        ' Show all hidden edges
        swView.HiddenEdges = vEdges

    End Sub

    Public swApp As SldWorks

End Class

```


Hide and Show Row (VBA)

This example shows how to hide and show a row in a general table.

```
'-----  
' Preconditions: Drawing document is open and  
'               contains a general table feature  
'               with at least three rows and that  
'               feature is selected.  
'  
' Postconditions: The third row of the general table is  
'               hidden and shown.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swGeneralTableFeature As SldWorks.GeneralTableFeature  
Dim vTables As Variant  
Dim swTable As SldWorks.TableAnnotation  
Dim boolstatus As Boolean  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swSelMgr = swModel.SelectionManager  
Set swModelDocExt = swModel.Extension  
  
' Interactively have selected a general table feature  
Set swGeneralTableFeature = swSelMgr.GetSelectedObject6(1, -1)  
  
Debug.Print "Nbr of table annotations: " & swGeneralTableFeature.GetTableAnnotationCount  
vTables = swGeneralTableFeature.GetTableAnnotations  
Set swTable = vTables(0)  
Debug.Print "Nbr of rows before hiding third third row: " & swTable.RowCount  
' Hide the third row  
swTable.RowHidden(2) = True  
Debug.Print "Nbr of rows after hiding third row: " & swTable.RowCount  
' Show the third row  
swTable.RowHidden(2) = False  
Debug.Print "Nbr of rows after showing third row: " & swTable.RowCount  
  
End Sub
```

Hide and Show Sketches Example (C#)

This example shows how to hide and show sketches in a model.

```
//-----
// Preconditions: Run the macro.
//
// Postconditions:
// 1. Opens a new part document and
//    creates two sketches.
// 2. Selects one sketch and hides it.
// 3. Examine the graphics area to verify
//    and press F5.
// 4. Selects the hidden sketch and shows
//    it.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace BlankUnBlankSketchesCSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchMgr = default(SketchManager);
            object[] sketchLines = null;
            SketchSegment swSketchSegment = default(SketchSegment);
            bool status = false;

            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2013\\templates\\Part.prt", (int)swDwgPaperSizes_e.swDwgPaperAsizeVertical, 0, 0);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSketchMgr = (SketchManager)swModel.SketchManager;

            // Sketch a rectangle
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swModel.ClearSelection2(true);
            sketchLines = (object[])swSketchMgr.CreateCornerRectangle(-0.0684166678777842, 0.0376953152008355, 0, -0.0273535635019471, 0.00483994917499331, 0);
            swModel.ClearSelection2(true);
            swSketchMgr.InsertSketch(true);

            // Sketch a circle
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swModel.ClearSelection2(true);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateCircle(0.044426, 0.079347, 0.0, 0.057359, 0.06229, 0.0);
            swModel.ClearSelection2(true);
            swSketchMgr.InsertSketch(true);

            swModel.ClearSelection2(true);

            // Select a sketch and hide it
            status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);
            swModel.BlankSketch();

            System.Diagnostics.Debugger.Break();
            // Examine the graphics area
            // to verify that the selected sketch
            // is hidden
            // Press F5

            swModel.ClearSelection2(true);

            // Select the just hidden sketch again
            // and show it
            status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);
            swModel.UnblankSketch();

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Hide and Show Sketches Example (VBA)

This example shows how to hide and show sketches in a model.

```
'-----  
' Preconditions:  
' 1. Open a new part document.  
' 2. Create multiple sketches.  
' 3. Select the sketches in the FeatureManager  
'    design tree or graphics area that you want  
'    to hide.  
' 4. Run the macro.  
'  
' Postconditions:  
' 1. Examine the graphics area.  
'    The selected sketches are hidden.  
' 2. Select the same sketches in the FeatureManager  
'    design tree, and press F5.  
' 3. Examine the graphics area to verify that  
'    the sketches are visible.  
'-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
  
    ' Hide the selected sketches  
    swModel.BlankSketch  
  
    Stop  
    ' Press F5  
  
    ' Select the just hidden sketches in the  
    ' FeatureManager design tree and show them  
    swModel.UnblankSketch
```

End Sub

Hide and Show Sketches Example (VB.NET)

This example shows how to hide and show sketches in a model.

```
.....
' Preconditions: Run the macro.
'
' Postconditions:
' 1. Opens a new part document and
'   creates two sketches.
' 2. Selects one sketch and hides it.
' 3. Examine the graphics area to verify
'   and press F5.
' 4. Selects the hidden sketch and shows
'   it.
.....
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchMgr As SketchManager
        Dim sketchLines As Object
        Dim swSketchSegment As SketchSegment
        Dim status As Boolean

        swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2013\templates\Part.prtdot", swDwgPaperSizes_e.swDwgPaperAsizeVertical, 0, 0)
        swModelDocExt = swModel.Extension
        swSketchMgr = swModel.SketchManager

        ' Sketch a rectangle
        status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        sketchLines = swSketchMgr.CreateCornerRectangle(-0.0684166678777842, 0.0376953152008355, 0, -0.0273535635019471, 0.00483994917499331, 0)
        swModel.ClearSelection2(True)
        swSketchMgr.InsertSketch(True)

        ' Sketch a circle
        status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        swSketchSegment = swSketchMgr.CreateCircle(0.044426, 0.079347, 0.0#, 0.057359, 0.06229, 0.0#)
        swModel.ClearSelection2(True)
        swSketchMgr.InsertSketch(True)

        swModel.ClearSelection2(True)

        ' Select a sketch and hide it
        status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        swModel.BlankSketch()

        Stop
        ' Examine the graphics area
        ' to verify that the selected sketch
        ' is hidden
        ' Press F5

        swModel.ClearSelection2(True)

        ' Select the just hidden sketch again
        ' and show it
        status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        swModel.UnblankSketch()

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

End Class

Hide Drawing Components Example (VBA)

This example shows how to hide the drawing components in the selected drawing view.

```
'-----
' Preconditions:
' 1. Open a drawing of an assembly.
' 2. Select a drawing view.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Hides the drawing components in the selected
'    drawing view.
' 2. Examine the drawing and the Immediate window.
'-----
Option Explicit

Sub ProcessDrawingComponent(swDrawComp As SldWorks.DrawingComponent, sPadStr As String)
    Dim vDrawCompChildArr As Variant
    Dim vDrawCompChild As Variant
    Dim swDrawCompChild As SldWorks.DrawingComponent

    Debug.Print sPadStr & swDrawComp.Name

    ' Does not affect root component
    swDrawComp.Visible = False
    vDrawCompChildArr = swDrawComp.GetChildren
    If Not IsEmpty(vDrawCompChildArr) Then
        For Each vDrawCompChild In vDrawCompChildArr
            Set swDrawCompChild = vDrawCompChild
            ProcessDrawingComponent swDrawCompChild, sPadStr + "  "
        Next
    End If
End Sub

Sub main()
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swDraw As SldWorks.DrawingDoc
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swView As SldWorks.View
    Dim swDrawComp As SldWorks.DrawingComponent
    Dim swComp As SldWorks.Component2
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel
    Set swSelMgr = swModel.SelectionManager
    Set swView = swSelMgr.GetSelectedObject6(1, -1)
    Set swDrawComp = swView.RootDrawingComponent
    Debug.Print "File = " & swModel.GetPathName
    Debug.Print "  " & swView.Name & " [" & swView.Type & "]"

    ProcessDrawingComponent swDrawComp, "  "
End Sub
```

Hide Feature in FeatureManager Design Tree Example (VBA)

This example shows how to hide the selected feature in the FeatureManager design tree.

```
'-----
' Preconditions:
' 1. Open a model document.
' 2. Select a feature.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Hides the selected feature in the FeatureManager
'    design tree, but it is still visible in the
'    graphics area.
' 2. Examine the Immediate window and graphics area.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As SldWorks.Feature
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)

    Debug.Print "Feature = " & swFeat.Name
    Debug.Print "  Type   = " & swFeat.GetTypeName
    Debug.Print "  Hidden  = " & swFeat.GetUIState(swIsHiddenInFeatureMgr)

    swFeat.SetUIState swIsHiddenInFeatureMgr, True

    swModel.EditRebuild3
    Debug.Print ""
    Debug.Print "  Hidden  = " & swFeat.GetUIState(swIsHiddenInFeatureMgr)

End Sub
```

Hide or Show All Types Example (VBA)

This example shows how to hide or show these types in a the current document:

- planes
 - curves
 - axes
 - sketches
 - temporary axes
 - all annotations (including model dimensions and feature dimensions)
- origins
 - points
 - coordinate systems
 - routing points

'-----

Option Explicit

Public Enum swUserPreferenceToggle_e
 swViewDisplayHideAllTypes = 198
End Enum

```
Sub main()  
    Dim swApp          As SldWorks.SldWorks  
    Dim swModel         As SldWorks.modelDoc  
    Dim bRet            As Boolean  
    Dim bShow           As Boolean  
    Dim nResponse       As Integer  
  
    nResponse = MsgBox("Hide all types?", vbYesNo)  
    If nResponse = vbYes Then  
        bShow = True  
    Else  
        bShow = False  
    End If  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    bRet = swModel.SetUserPreferenceToggle(swViewDisplayHideAllTypes, bShow): Debug.Assert bRet  
End Sub  
  
'-----
```

Hide or Show First Column in Hole Table Example (C#)

This example shows how to hide the first column in a hole table.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\simplehole.slddrw.
// 2. Select the hole table feature in the FeatureManager
//    design tree.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Hides the first column.
// 2. Examine the hole table and Immediate window.
//
// NOTE: Because the drawing is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace ColumnHiddenTableAnnotation_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            object[] swHoleTableAnnotation = new object[1];
            TableAnnotation swTableAnnotation = default(TableAnnotation);
            Annotation swAnnotation = default(Annotation);
            HoleTable swHoleTable = default(HoleTable);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            if ((swSelMgr.GetSelectedObjectCount() == 0) || (swSelMgr.GetSelectedObjectType(1) ==
(int)swSelectType_e.swSelHOLETABLEFEATS))
            {
                System.Windows.Forms.MessageBox.Show("Please select a hole table feature in the FeatureManager design tree.");
                return;
            }
            swHoleTable = (HoleTable)swSelMgr.GetSelectedObject6(1, -1);
            swHoleTableAnnotation = (object[])swHoleTable.GetTableAnnotations();
            swTableAnnotation = (TableAnnotation)swHoleTableAnnotation[0];
            swAnnotation = (Annotation)swTableAnnotation.GetAnnotation();
            // If column hidden, then show it
            if ((swTableAnnotation.get_ColumnHidden(0)))
            {
                Debug.Print(" First column hidden before API call: " + swTableAnnotation.get_ColumnHidden(0));
                swTableAnnotation.set_ColumnHidden(0, false);
                Debug.Print(" First column hidden after API call: " + swTableAnnotation.get_ColumnHidden(0));
            }
            else
            {
                // If column shown, then hide it
                Debug.Print(" First column hidden before API call: " + swTableAnnotation.get_ColumnHidden(0));
                swTableAnnotation.set_ColumnHidden(0, true);
                Debug.Print(" First column hidden after API call: " + swTableAnnotation.get_ColumnHidden(0));
            }
        }
    }
    public SldWorks swApp;
}
}
```

Hide or Show First Column in Hole Table Example (VBA)

This example shows how to hide the first column in a hole table.

```
' -----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\simplehole.slddrw.
' 2. Select the hole table feature in the FeatureManager
'    design tree.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Hides the first column.
' 2. Examine the hole table and Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
' -----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swHoleTableAnnotation As SldWorks.HoleTableAnnotation
    Dim swTableAnnotation As SldWorks.TableAnnotation
    Dim swAnnotation As SldWorks.Annotation
    Dim swHoleTable As SldWorks.HoleTable

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager

    If (swSelMgr.GetSelectedObjectCount = 0 Or Not (swSelMgr.GetSelectedObjectType(1) = swSelHOLETABLEFEATS)) Then
        MsgBox "Please select a hole table feature in the FeatureManager design tree."
        Exit Sub
    End If

    Set swHoleTable = swSelMgr.GetSelectedObject6(1, -1)
    Set swHoleTableAnnotation = swHoleTable.GetTableAnnotations(0)
    Set swTableAnnotation = swHoleTableAnnotation
    Set swAnnotation = swTableAnnotation.GetAnnotation

    ' If first column is hidden, then show it
    If (swTableAnnotation.ColumnHidden(0)) Then
        Debug.Print " First column hidden before API call: " & swTableAnnotation.ColumnHidden(0)
        swTableAnnotation.ColumnHidden(0) = False
        Debug.Print " First column hidden after API call: " & swTableAnnotation.ColumnHidden(0)
    Else
        ' If first column is shown, then hide it
        Debug.Print " First column hidden before API call: " & swTableAnnotation.ColumnHidden(0)
        swTableAnnotation.ColumnHidden(0) = True
        Debug.Print " First column hidden after API call: " & swTableAnnotation.ColumnHidden(0)
    End If

End Sub
```

Hide or Show First Column in Hole Table Example (VB.NET)

This example shows how to hide the first column in a hole table.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\simplehole.slddrw.
' 2. Select the hole table feature in the FeatureManager
'    design tree.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Hides the first column.
' 2. Examine the hole table and Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics
```

Partial Class SolidWorksMacro

```
    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swHoleTableAnnotation As HoleTableAnnotation
        Dim swTableAnnotation As TableAnnotation
        Dim swAnnotation As Annotation
        Dim swHoleTable As HoleTable

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager

        If (swSelMgr.GetSelectedObjectCount = 0 Or Not (swSelMgr.GetSelectedObjectType(1) = swSelectType_e.swSelHOLETABLEFEATS))
Then
            MsgBox("Please select a hole table feature in the FeatureManager design tree.")
            Exit Sub
        End If

        swHoleTable = swSelMgr.GetSelectedObject6(1, -1)
        swHoleTableAnnotation = swHoleTable.GetTableAnnotations(0)
        swTableAnnotation = swHoleTableAnnotation
        swAnnotation = swTableAnnotation.GetAnnotation

        ' If first column hidden, then show it
        If (swTableAnnotation.ColumnHidden(0)) Then
            Debug.Print(" First column hidden before API call: " & swTableAnnotation.ColumnHidden(0))
            swTableAnnotation.ColumnHidden(0) = False
            Debug.Print(" First column hidden after API call: " & swTableAnnotation.ColumnHidden(0))
        Else
            ' If first column shown, then hide it
            Debug.Print(" First column hidden before API call: " & swTableAnnotation.ColumnHidden(0))
            swTableAnnotation.ColumnHidden(0) = True
            Debug.Print(" First column hidden after API call: " & swTableAnnotation.ColumnHidden(0))
        End If

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Hide or Show First Row in Bill of Materials Example (VBA)

This example shows how to hide the first row in a Bill of Materials if it is visible, or show the first row in a Bill of Materials if it is hidden.

```
'-----
'
' Preconditions: Drawing document is open and contains
'               a Bill of Materials with at least one
'               row. Any row in the Bill of Materials is
'               selected.
'
' Postconditions: The first row in the Bill of Materials
'                is hidden if it was visible or the first
'                row in the Bill of Materials is visible
'                if it was hidden.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swBomTable As SldWorks.BomTableAnnotation
Dim swAnnotationTable As SldWorks.TableAnnotation
```

Sub main()

```
Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swSelMgr = swModel.SelectionManager
Set swBomTable = swSelMgr.GetSelectedObject6(1, -1)
Set swAnnotationTable = swBomTable

If swBomTable.RowHidden(1) Then
    swBomTable.RowHidden(1) = False
Else
    swBomTable.RowHidden(1) = True
End If
```

End Sub

Hide the FeatureManager Design Tree Example (C#)

This example shows how to hide the FeatureManager design tree.

```
//-----  
// Preconditions: Open a model document.  
//  
// Postconditions:  
// 1. Hides the FeatureManager design tree.  
// 2. Examine SOLIDWORKS Design.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
namespace HideFeatureManager_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
  
        void Main()  
        {  
  
            ModelDoc2 swModel;  
            ModelDocExtension swModDocExt;  
            bool bRet;  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swModDocExt = swModel.Extension;  
            bRet = swModDocExt.HideFeatureManager(true);  
  
        }  
  
        public SldWorks swApp;  
  
    }  
}
```

Hide the FeatureManager Design Tree Example (VBA)

This example shows how to hide the FeatureManager design tree.

```
'-----  
' Preconditions: Open a model document.  
'  
' Postconditions:  
' 1. Hides the FeatureManager design tree.  
' 2. Examine SOLIDWORKS Design.  
'-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swModDocExt As SldWorks.ModelDocExtension  
    Dim bRet As Boolean
```

```
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModDocExt = swModel.Extension  
    bRet = swModDocExt.HideFeatureManager(True)
```

End Sub

Hide the FeatureManager Design Tree Example (VB.NET)

This example shows how to hide the FeatureManager design tree.

```
'-----  
' Preconditions: Open a model document.  
'  
' Postconditions:  
' 1. Hides the FeatureManager design tree.  
' 2. Examine SOLIDWORKS Design.  
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```
        Dim swModel As ModelDoc2
```

```
        Dim swModDocExt As ModelDocExtension
```

```
        Dim bRet As Boolean
```

```
        swModel = swApp.ActiveDoc
```

```
        swModDocExt = swModel.Extension
```

```
        bRet = swModDocExt.HideFeatureManager(True)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Ignore Feature Colors Example (VBA)

This example shows how to change a user preference option so that feature colors are ignored in parts. This option setting, `swIgnoreFeatureColors`, can be true or false and can be different for each document. It is not a system-level setting; it is a document-level setting. Therefore, you must set this value for each document.

By default, when a document is created, `swIgnoreFeatureColors` is false.

```
'-----
' Preconditions: Open a part document.
'
' Postconditions:
' 1. Gets current setting for swIgnoreFeatureColors and
'    pops up a message box. Click OK to close the message
'    box.
' 2. Pops up a message box prior to setting swIgnoreFeatureColors
'    to true. Click OK to close the message box.
' 3. Sets swIgnoreFeatureColors to true and pops up a message
'    box indicating setting swIgnoreFeatureColors to true
'    is successful. Click OK to close the message box.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim res, val As Boolean
Dim msg1 As String

Sub main()

    Set swApp = CreateObject("SldWorks.Application")
    Set Part = swApp.ActiveDoc

    ' Exit if no model is active
    If Part Is Nothing Then
        swApp.SendMsgToUser "Open a model document and run the macro again."
        Exit Sub
    End If

    ' Get current setting
    val = Part.GetUserPreferenceToggle(swIgnoreFeatureColors)

    ' Create message
    msg1 = "swIgnoreFeatureColors is currently set to " & val & "."

    ' Send message to user
    swApp.SendMsgToUser msg1

    ' Change setting to true, which indicates to ignore feature colors
    swApp.SendMsgToUser "Setting swIgnoreFeatureColors to True."
    res = Part.SetUserPreferenceToggle(swIgnoreFeatureColors, True)

    ' Verify if successful    If (res = True) Then
        swApp.SendMsgToUser "Setting swIgnoreFeatureColors to True successful."
    Else
        swApp.SendMsgToUser "Error! Setting swIgnoreFeatureColors to True not successful."
    End If

End Sub
```